

homepage_only.js - Complete Documentation

File Location: src/js/homepage_only.js
Purpose: Event loading for public landing page (non-logged-in users)
Dependencies: modal1.js
Used by: index.php (public homepage)

Overview

The homepage_only.js file handles event loading specifically for the public landing page where users are not logged in. It displays upcoming and popular events, but forces users to log in before they can view event details. This encourages user registration.

Functions

loadUpcomingEventsforOther()

Purpose: Loads upcoming events for public visitors.

Parameters: None
Returns: void

What it does:

- Similar to loadUpcomingEvents() in both_homepage.js
- Fetches from ../php/get_upcoming_events.php (different path)
- Displays in #eventsGrid
- Shows upcoming events to encourage users to register

Code:

```
function loadUpcomingEventsforOther() {  
  fetch('../php/get_upcoming_events.php')  
    .then(res => res.json())  
    .then(events => {  
      const grid = document.getElementById('eventsGrid');  
      if (!grid) return;  
      grid.innerHTML = events.map(event => `  
        <div class="event-card">  
          <div class="event-card-header">  
            <h3>${event.ename}</h3>  
            <span class="event-category">${event.category ? event.category.charAt(0).toUpperCase() + event.category.slice(1)}</span>  
            <span class="event-card-date">${event.date}</span>  
          </div>  
          <div class="event-card-body">  
            <p>${event.description.substring(0, 100)}...</p>  
          </div>  
          <div class="event-card-footer">  
            <button class="btn btn-more" onclick="viewEventDetails(${event.event_id})">View Details</button>  
          </div>  
        </div>  
      `).join('');  
    });  
}
```

Path Difference:

- Admin/Student dashboards: ../../php/... (from admin/ or student/ subdirectories)
- Public homepage: ../php/... (from root frontends directory)

User Experience:

```
User visits index.php
↓
loadUpcomingEventsforOther() loads
↓
Displays upcoming event cards
↓
User sees interesting events
↓
Encourages registration
```

loadPopularEvents()

Purpose: Loads popular/trending events for public visitors.

Parameters: None

Returns: void

What it does:

- Fetches from ../php/get_popular_events.php
- Shows most popular events (sorted by participant count)
- Displays in #popularEventsGrid
- Creates social proof: "See what others are enjoying"

Code:

```
function loadPopularEvents() {
    fetch('../php/get_popular_events.php')
        .then(res => res.json())
        .then(events => {
            const grid = document.getElementById('popularEventsGrid');
            if (!grid) return;
            grid.innerHTML = events.map(event => `
                <div class="event-card">
                    <div class="event-card-header">
                        <h3>${event.ename}</h3>
                        <span class="event-category">${event.category ? event.category.charAt(0).toUpperCase() + event.category.slice(1)}</span>
                        <span class="event-card-date">${event.date}</span>
                    </div>
                    <div class="event-card-body">
                        <p>${event.description.substring(0, 100)}...</p>
                    </div>
                    <div class="event-card-footer">
                        <button class="btn btn-more" onclick="viewEventDetails(${event.event_id})">View Details</button>
                    </div>
                </div>
            `).join('');
        });
}
```

Why Show Popular Events?

- ☐ Creates FOMO (Fear Of Missing Out)
- ☐ Shows active community
- ☐ Encourages sign-ups
- ☐ Highlights best events

viewEventDetails(eventId)

Purpose: Redirect non-logged-in users to login page.

Parameters:

- eventId (integer) - Event ID (not used for public users)

Returns: void

What it does:

- For public users, clicking event details redirects to login
- Prevents unauthorized access to event information
- Encourages user registration
- **Shows:** `window.location.href = 'login.html'`

Code:

```
function viewEventDetails(eventId) {  
    window.location.href = 'login.html';  
}
```

User Experience:

```
Public user visits index.php  
    ↓  
Sees interesting events  
    ↓  
Clicks "View Details"  
    ↓  
Redirected to login.html  
    ↓  
Prompted to create account  
    ↓  
After login, redirected to dashboard  
    ↓  
Full access to event details
```

Security Approach:

- ☐ Doesn't expose full event details to non-users
- ☐ Gently encourages registration
- ☐ Prevents API abuse (unauthorized event data access)

Page Initialization

DOMContentLoaded Event

```
document.addEventListener('DOMContentLoaded', function() {  
    loadUpcomingEventsforOther();  
    loadPopularEvents();  
});
```

Execution Sequence:

```
Page starts loading index.php
↓
HTML parsed
↓
CSS loaded
↓
JavaScript files loaded
↓
DOMContentLoaded event fires
↓
Both event-loading functions called
↓
AJAX requests sent to PHP
↓
Event data fetched
↓
HTML generated and injected
↓
Page displays populated event grids
```

Why Wait for DOMContentLoaded?

- Ensures #eventsGrid and #popularEventsGrid elements exist
- DOM is fully parsed before manipulating it
- Prevents "element not found" errors

HTML Structure Required

Event Grid Containers

```
<!-- Upcoming Events Section -->
<section class="events-section">
  <h2>Upcoming Events</h2>
  <div id="eventsGrid" class="events-grid">
    <!-- Populated by loadUpcomingEventsforOther() -->
  </div>
</section>

<!-- Popular Events Section -->
<section class="events-section">
  <h2>Popular Events</h2>
  <div id="popularEventsGrid" class="events-grid">
    <!-- Populated by loadPopularEvents() -->
  </div>
</section>
```

CSS Grid Styling

```
.events-grid {
  display: grid;
  grid-template-columns: repeat(auto-fill, minmax(300px, 1fr));
  gap: 20px;
  padding: 20px;
}

@media (max-width: 768px) {
  .events-grid {
    grid-template-columns: 1fr;
  }
}
```

Comparison: Public vs. Logged-In

Aspect	Public (index.php)	Admin/Student (dashboards)
View Details	Redirect to login	Opens modal with full info
Path	../php/...	../../php/...
File	homepage_only.js	both_homepage.js
Sections	Upcoming + Popular	Upcoming + Recent + History
Access	Limited preview	Full access

User Journey

Discovery Funnel

```
Public Homepage (index.php)
↓
Browse Upcoming Events
↓
Browse Popular Events
↓
Click "View Details"
↓
Redirected to Login
↓
No account? → Sign Up
↓
Account Created
↓
Logged into Dashboard
↓
Full event access
↓
Register for events
```

Error Handling

Current Implementation

```
.then(res => res.json())
.then(events => {
  // ...
});
// No error handling - silent failure
```

Improved Version

```
function loadUpcomingEventsforOther() {
  const grid = document.getElementById('eventsGrid');
  if (!grid) {
    console.error('Element #eventsGrid not found');
    return;
  }

  // Show loading state
  grid.innerHTML = '<p class="loading">Loading events...</p>';

  fetch('../php/get_upcoming_events.php')
    .then(res => {
      if (!res.ok) throw new Error(`HTTP ${res.status}`);
      return res.json();
    })
    .then(events => {
      if (!Array.isArray(events) || events.length === 0) {
        grid.innerHTML = '<p class="no-events">No upcoming events</p>';
        return;
      }
      grid.innerHTML = events.map(event => /* ... */).join('');
    })
    .catch(err => {
      console.error('Failed to load events:', err);
      grid.innerHTML = '<p class="error">Failed to load events. Please try again.</p>';
    });
}
```

Integration Strategy

Why Separate File?

Rather than including `both_homepage.js` in public pages, a separate file:

- ☐ Reduces payload for public users
- ☐ Prevents accidental exposure of admin functions
- ☐ Makes code intentions clearer
- ☐ Easier to maintain different logic

File Organization

```
src/js/
├─ modal1.js      (Shared modal system)
├─ valid1.js      (Shared validation)
├─ toggle_modes.js (Shared theme)
├─ both_homepage.js (Logged-in dashboards)
└─ homepage_only.js (Public page only)
```

Marketing Use Case

The public homepage acts as a **lead magnet**:

1. **Awareness:** Show interesting events
2. **Interest:** Highlight popular/trending events
3. **Desire:** Tease details (requires login)
4. **Action:** User registers to see more

This funnel converts:

- ☐ Casual browsers → Account holders
- ☐ Interested users → Active participants

Performance Notes

AJAX Requests

```
fetch('../php/get_upcoming_events.php') // 1 request
fetch('../php/get_popular_events.php') // 1 request
// Total: 2 requests, can be optimized to 1
```

Optimization Ideas

```
// Combine into single request
async function loadPublicEvents() {
  const data = await fetch('../php/get_public_events.php')
    .then(r => r.json());

  renderUpcoming(data.upcoming);
  renderPopular(data.popular);
}
```

Summary

homepage_only.js provides:

Feature	Purpose
loadUpcomingEventsforOther()	Show future events to public
loadPopularEvents()	Show trending events
viewEventDetails()	Force login for details
DOMContentLoaded	Initialize on page load

Creates an enticing public-facing homepage that encourages user registration.

Last Updated: May 22, 2026

File Size: ~30 lines of code

Complexity: Low

Criticality: Medium (user acquisition)